

Quant Sentiment Engine

NLP Pipeline for Earnings-Call & Financial Document Sentiment Scoring

Tech Stack: Python · spaCy · Hugging Face · FastAPI · PostgreSQL

Version v1.0 · June 2026 · Internal Engineering Specification

Table of Contents

#	Section	Description
1	Project Overview	Goals, scope, stakeholders, success criteria
2	System Architecture	Component diagram, data flow, deployment topology
3	Data Ingestion	Sources, connectors, scheduling, rate-limiting
4	NLP Pipeline	Pre-processing, models, sentiment scoring, backtesting
5	API Specification	FastAPI endpoints, auth, request/response schemas
6	Database Schema	PostgreSQL tables, indexes, migrations
7	Backtesting Module	S&P; 500 sector factor construction & evaluation
8	Infrastructure & DevOps	Docker, CI/CD, monitoring, secrets
9	Testing Strategy	Unit, integration, regression, load tests
10	Error Handling & Logging	Retry logic, alerting, audit trail

1. Project Overview

The **Quant Sentiment Engine** is an end-to-end NLP pipeline that ingests earnings-call transcripts, 10-K/10-Q filings, and financial news articles, then produces *tradeable sentiment factor scores* per S&P 500 sector. These scores feed downstream quant strategies via a REST API.

Owner	Quant Research Team
Document Type	Software Engineering Specification
Status	Active Development
Target Release	Q3 2026
Primary Language	Python 3.11+
Key Libraries	spaCy 3.7, Hugging Face Transformers 4.40, FastAPI 0.111, PostgreSQL 16

1.1 Goals

- Ingest & parse $\geq 5,000$ earnings transcripts per quarter from SEC EDGAR and vendor APIs
- Score each document across 6 sentiment dimensions: overall tone, forward guidance, risk language, M&A signals, capex intent, labour sentiment
- Produce sector-level factor scores updated daily with < 2 -hour latency from source release
- Backtest sentiment factors against S&P 500 sector returns over 10-year window
- Expose all scores and metadata via a low-latency REST API (p95 < 80 ms)

1.2 Out of Scope

- Real-time order execution or broker connectivity
- Non-English documents (Phase 1)
- Alternative data sources beyond text (satellite imagery, credit-card data)

1.3 Success Criteria

Metric	Target	Measurement
Sentiment label accuracy	$\geq 82\%$ F1 vs human annotators	Weekly eval on 200-sample held-out set

API p95 latency	< 80 ms	Prometheus / Grafana dashboard
Pipeline uptime	$\geq 99.5\%$ (monthly)	PagerDuty on-call alerts
Factor IC (Information Coefficient)	> 0.04 rolling 12-month	Backtest module nightly run
Backtest Sharpe (long-short)	> 0.8 annualised	Backtest report PDF emailed weekly

2. System Architecture

The pipeline is composed of four loosely-coupled services communicating via async queues and a shared PostgreSQL database. All services run as Docker containers orchestrated by Docker Compose (dev) or Kubernetes (prod).

2.1 Component Overview

Component	Responsibility	Technology
Ingestion Service	Polls EDGAR, NewsAPI, Seeking Alpha; stores raw docs	Python, APScheduler, httpx
NLP Worker	Pre-processes text, runs transformer models, writes scores	spaCy, HuggingFace, Celery
Score API	Serves sentiment scores to downstream consumers	FastAPI, Redis cache
Backtest Engine	Constructs factor returns, computes IC/IR/Sharpe	Python, pandas, yfinance
Postgres DB	Persistent store for documents, scores, metadata	PostgreSQL 16, Alembic
Task Queue	Distributes NLP jobs to worker pool	Redis + Celery
Monitoring	Metrics, alerts, tracing	Prometheus, Grafana, Sentry

2.2 Data Flow

- **Ingest:** Scheduler triggers ingestion workers every 15 min (news) and nightly (filings)
- **Queue:** Each new raw document emits a Celery task to the NLP queue
- **Process:** NLP workers pull tasks, run the full scoring pipeline, write results to Postgres
- **Cache:** FastAPI reads from Redis (TTL 5 min); cache-miss falls back to Postgres
- **Backtest:** Nightly cron reads scores + price data, writes factor returns to backtest schema

3. Data Ingestion

3.1 Sources

Source	Docs	Connector	Frequency
SEC EDGAR	10-K, 10-Q, 8-K filings	EDGAR Full-Text Search API	Nightly 02:00 UTC
Seeking Alpha	Earnings transcripts	Rapid API wrapper	On-event + nightly catch-up
NewsAPI	Financial news articles	REST /v2/everything	Every 15 min
Refinitiv Eikon	Premium news & transcripts	Eikon Data API (optional)	Real-time (optional tier)

3.2 Ingestion Worker — Key Implementation Notes

- All HTTP calls wrapped in **httpx.AsyncClient** with exponential back-off (max 5 retries, base 2 s)
- Rate-limit headers respected; token-bucket limiter enforced client-side before each request
- Raw HTML stripped via **trafilatura**; PDF filings parsed via **pdfplumber**
- Deduplication: SHA-256 hash of (source, document_id) checked against **raw_documents.content_hash** before insert
- Failed ingestion jobs written to **dead_letter_queue** table with full traceback for manual review

// Example: EDGAR async connector

```
# ingestion/edgar_connector.py
async def fetch_filings(ticker: str, form_type: str, since: date) -> list[RawDoc]:
    url = f'https://efits.sec.gov/LATEST/search-index?q={ticker}&dateRange=custom'
    async with AsyncClient(timeout=30) as client:
        resp = await client.get(url, params={'startdt': since.isoformat(),
        'forms': form_type})
        resp.raise_for_status()
        hits = resp.json()['hits']['hits']
        return [_parse_hit(h) for h in hits]
```

4. NLP Pipeline

4.1 Pre-processing Steps

- **Language detection** — langdetect; non-English docs flagged, not processed in Phase 1
- **Boilerplate removal** — SEC legal disclaimers, standard risk-factor headers stripped via regex rule set
- **Sentence segmentation** — spaCy `en_core_web_trf` pipeline
- **Named-entity recognition** — spaCy NER tags ORG, MONEY, PERCENT, DATE for downstream feature extraction
- **Chunking** — documents > 512 tokens split into overlapping 384-token windows (overlap 128) for transformer input

4.2 Scoring Models

Dimension	Model	Output
Overall tone	ProsusAI/finbert (fine-tuned)	[-1, 1] continuous score
Forward guidance	Custom classifier on FinBERT embeddings	Positive / Neutral / Negative
Risk language density	spaCy rule-based + TF-IDF risk lexicon	[0, 1] density score
M&A signals	Zero-shot NLI (facebook/bart-large-mnli)	P(acquisition intent)
Capex intent	Zero-shot NLI	P(capex increase)
Labour sentiment	Fine-tuned DistilBERT on HR corpus	[-1, 1] score

4.3 Sector Aggregation

Individual document scores are aggregated to S&P 500 GICS sector level using a market-cap-weighted mean. Weights sourced daily from the **sector_weights** table (populated by ingestion job from SPDR ETF holdings).

```
// Sector aggregation (simplified)
# nlp/aggregation.py

def aggregate_sector_scores(scores: list[DocScore], weights: dict[str, float]) -> SectorScore:
    df = pd.DataFrame([s.dict() for s in scores])
    df['weight'] = df['ticker'].map(weights).fillna(0)
    df['weighted_tone'] = df['overall_tone'] * df['weight']
    sector_score = df.groupby('sector')['weighted_tone'].sum() /
    df.groupby('sector')['weight'].sum()
```

```
return SectorScore(scores=sector_score.to_dict(), as_of=date.today())
```

5. API Specification

All endpoints are served by FastAPI under base path **/api/v1**. Authentication uses **Bearer JWT** tokens issued by the internal auth service.

Method	Endpoint	Description	Auth
GET	/scores/sector/{sector}	Latest sector sentiment scores	Bearer
GET	/scores/ticker/{ticker}	Latest scores for a single ticker	Bearer
GET	/scores/history	Historical scores with date range filter	Bearer
GET	/documents/{doc_id}	Raw document metadata + parsed text	Bearer
GET	/backtest/summary	Latest backtest IC, Sharpe, factor returns	Bearer + Admin
POST	/ingest/manual	Manually trigger ingestion for a ticker	Bearer + Admin
GET	/health	Liveness + readiness probe	None

5.1 Response Schema — GET /scores/sector/{sector}

// JSON response

```
{
  "sector": "Information Technology",
  "as_of": "2026-06-26",
  "overall_tone": 0.34,
  "forward_guidance": "positive",
  "risk_density": 0.18,
  "ma_signal": 0.07,
  "capex_intent": 0.22,
  "labour_sentiment": 0.11,
  "doc_count": 143,
  "coverage_pct": 91.2
}
```

6. Database Schema

6.1 Core Tables

```
-- PostgreSQL DDL (abridged)
```

```
-- raw_documents: stores every ingested document
```

```
CREATE TABLE raw_documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  source TEXT NOT NULL, -- 'edgar' | 'seeking_alpha' | 'newsapi'  
  ticker TEXT,  
  doc_type TEXT, -- '10-K' | 'transcript' | 'news'  
  published_at TIMESTAMPTZ NOT NULL,  
  content_hash TEXT UNIQUE NOT NULL, -- SHA-256 for dedup  
  raw_text TEXT NOT NULL,  
  parsed_text TEXT,  
  word_count INT,  
  ingested_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- sentiment_scores: output of NLP pipeline per document
```

```
CREATE TABLE sentiment_scores (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  doc_id UUID REFERENCES raw_documents(id),  
  scored_at TIMESTAMPTZ DEFAULT NOW(),  
  model_version TEXT NOT NULL,  
  overall_tone FLOAT,  
  fwd_guidance TEXT,  
  risk_density FLOAT,  
  ma_signal FLOAT,  
  capex_intent FLOAT,  
  labour_score FLOAT  
);
```

```
-- sector_scores: daily aggregated sector factors
```

```
CREATE TABLE sector_scores (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  sector TEXT NOT NULL,  
  score_date DATE NOT NULL,  
  overall_tone FLOAT,  
  doc_count INT,  
  UNIQUE(sector, score_date)  
);
```

6.2 Indexes

- **raw_documents**: index on (ticker, published_at DESC), index on content_hash
- **sentiment_scores**: index on (doc_id), index on scored_at
- **sector_scores**: index on (sector, score_date DESC)

6.3 Migrations

- Managed via **Alembic**; migration scripts live in **db/migrations/**
- All schema changes applied via CI/CD pipeline before new service version deploys
- Rollback scripts mandatory for every migration

7. Backtesting Module

The backtesting module constructs a long-short factor portfolio based on sector sentiment scores and evaluates performance against S&P 500 sector ETF returns.

7.1 Factor Construction

- Daily sector scores normalised (z-score, rolling 252-day window) to form factor signal
- Top tercile sectors go long, bottom tercile go short; middle tercile excluded
- Portfolio rebalanced weekly (Wednesday close); equal-weight within long/short baskets
- Transaction costs modelled at 5 bps per side

7.2 Evaluation Metrics

Metric	Formula	Target
IC (Rank)	Spearman corr(signal_t, return_t+1w)	> 0.04
IR (Information Ratio)	mean(IC) / std(IC)	> 0.5
Annualised Sharpe	mean(r) / std(r) * sqrt(52)	> 0.8
Max Drawdown	Peak-to-trough cumulative return	< 25%
Hit Rate	% weeks with positive L/S return	> 52%

8. Infrastructure & DevOps

8.1 Docker Services

```
// docker-compose.yml (abridged)
services:
  postgres: image: postgres:16-alpine
  redis: image: redis:7-alpine
  ingestion: build: ./services/ingestion
  nlp_worker: build: ./services/nlp # scale: 4 replicas in prod
  api: build: ./services/api # expose :8000
  backtest: build: ./services/backtest # cron-only
  prometheus: image: prom/prometheus
  grafana: image: grafana/grafana
```

8.2 CI/CD — GitHub Actions

- **On PR:** lint (ruff), type-check (mypy), unit tests, docker build cache validation
- **On merge to main:** full test suite, build + push Docker images to ECR, deploy to staging
- **On release tag:** deploy to production with blue-green rollout, smoke tests, auto-rollback on failure

8.3 Secrets Management

- All secrets (API keys, DB passwords) stored in **AWS Secrets Manager**
- Retrieved at container startup via **aws-secretsmanager-agent** sidecar; never in .env files
- Rotation enforced every 90 days via automated Lambda function

9. Testing Strategy

Level	Framework	Coverage Target	Key Scenarios
Unit	pytest	$\geq 85\%$	Scorer logic, dedup hash, aggregation math
Integration	pytest + testcontainers	$\geq 70\%$	API endpoints, Celery task dispatch, DB writes
Model Regression	Custom eval harness	Weekly	F1 $\geq 82\%$ on held-out annotation set
Load	Locust	Before release each	500 req/s sustained, p95 < 80 ms
Contract	Schemathesis	Every PR	OpenAPI schema compliance, fuzz inputs

10. Error Handling & Logging

10.1 Retry Policy

- Ingestion HTTP errors: exponential back-off, max 5 retries, jitter $\pm 20\%$
- NLP model inference errors: retry once on same worker; on second failure, route to **error_tasks** queue
- DB write failures: retry 3× with 500 ms delay; on exhaustion write to **dead_letter_queue**

10.2 Structured Logging

```
// JSON log format (all services)
{"timestamp": "2026-06-26T10:15:30Z", "level": "ERROR", "service": "nlp_worker",
"event": "scoring_failed", "doc_id": "abc-123", "ticker": "MSFT",
"error": "CUDA OOM", "traceback": "...", "worker_id": "worker-3"}
```

10.3 Alerting Runbook

- **NLP queue depth > 10k**: Scale NLP worker replicas; check GPU availability
- **Model F1 drops > 5pp**: Trigger model retraining pipeline; roll back to previous model version
- **API error rate > 1%**: Check DB connection pool; restart API pods if necessary
- **Ingestion source 4xx/5xx > 20%**: Check API key rotation; notify data vendor support